SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – ERROR ANALYSIS

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	T. Kiran Kumar	**Reg. No.:**	192472321
Section / Batch:	CSE(Ai)	**Date:**	19-08-2026

Code of Conduct

I T . Kiran Kumar (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: T. Kiran Kumar

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational Morphology	**Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.**	**Q1**
Inflectional Morphology and Derivational Morphology	**Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.**	**Q2**
Finite-State Morphological Parsing	**Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.**	**Q3**
Finite-State Morphological Parsing	**Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.**	**Q4**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a Porter Stemmer implementation for words such as *connected, connection, connecting,* and *connectivity*; identify stemming-rule errors and correct the implementation.**	**Q5**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a morphological normalization program that processes *studies, studied, studying,* and *study*; identify incorrect stemming outputs and modify the code to produce consistent stems.**	**Q6**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.**	**Q7**

Annexure A – Error Analysis

Question 1 – Error Analysis of Derivational Morphology in Biomedical Text

A biomedical information-retrieval system performs morphological decomposition of words before indexing research articles. The system incorrectly analyzes the words:

1. **treatment**
2. **treatable**
3. **retreatment**
4. **treated**
5. **untreated**

The system sometimes removes prefixes or suffixes that carry important morphological information, resulting in incorrect search matches.

Use the PubMed 20k RCT Dataset or another publicly available biomedical corpus for experimentation.

Tasks

1. **Identify words that produce incorrect morphological analyses.**
2. **Decompose the selected words into prefixes, roots, and suffixes.**
3. **Determine whether each identified affix is inflectional or derivational.**
4. **Explain the root cause of the incorrect morphological analysis.**
5. **Propose a corrected morphological-analysis strategy.**
6. **Compare the original and corrected analyses.**
7. **Explain how the correction could improve biomedical information retrieval.**

Question 2 – Error Analysis of Finite-State Morphological Parsing

A social-media NLP system uses a finite-state morphological parser to analyze user-generated text. The parser produces incorrect analyses for words such as:

1. **replayed**
2. **unhappier**
3. **disconnected**
4. **players**
5. **restarting**
6. **unreadable**

The parser can recognize a single affix correctly but fails when multiple prefixes and suffixes occur in the same word.

Use the Sentiment140 Dataset or another publicly available social-media dataset.

Tasks

1. **Identify words that produce incorrect morphological analyses.**
2. **Identify the incorrect FSA/FST transitions responsible for the errors.**
3. **Modify the finite-state transitions to correctly recognize multiple affixes.**
4. **Execute the corrected parser on the selected dataset.**
5. **Compare the parser output before and after correction.**
6. **Calculate the parsing accuracy before and after correction.**
7. **Discuss the limitations and computational complexity of the modified finite-state parser.**

Question 3 – Identify the Error, Rectify It, and Analyze the Output

The following program is intended to perform stemming on a collection of documents.

```
from nltk.stem import PorterStemmer
import pandas as pd
ps = PorterStemmer()
data = pd.read_csv("news.csv")
data["Processed"] = data["Text"].apply(ps.stem)
print(data["Processed"].head())
```

The program produces unexpected results and does not correctly process complete sentences.

Tasks

1. **Identify the programming and preprocessing errors.**
2. **Explain why the output is incorrect.**
3. **Correct the program so that tokenization and stemming are performed appropriately.**
4. **Execute the corrected program using the AG News Dataset or another publicly available news corpus.**
5. **Compare:**
   - **Original text**
   - **Tokens**
   - **Stemmed tokens**
6. **Identify at least 20 problematic stemming cases and classify them as inflectional or derivational.**

7. Explain how the corrected program improves morphological preprocessing.

Question 4 – Error Analysis of a Finite-State Morphological Parser

The following Python program is intended to identify plural nouns:

```
def parser(word):
if word.endswith("s"):
    return word[:-2], "Plural Noun"
else:
    return word, "Singular"

words = [
  "cars",
  "boxes",
  "cities",
  "children",
  "books"
]

for w in words:
  print(w, "->", parser(w))
```

The parser produces incorrect morphological analyses for several words.

Tasks

1. Identify all logical errors in the program.
2. Explain why the parser fails for different plural formation rules.
3. Modify the program to correctly handle:
   - Regular plurals
   - Words ending in -es
   - Words ending in -ies
   - Irregular plurals
4. Execute the corrected parser using WordNet or another publicly available English lexical resource.
5. Compare the original and corrected outputs.
6. Identify the limitations of the rule-based finite-state approach.
7. Explain how the corrected parser improves morphological analysis.

Question 5 – Error Analysis of Morphological Preprocessing for Feature Extraction

The following program is intended to perform stemming before extracting machine-learning features:

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
     "connected connection connecting connectivity",
    "studies studied studying study",
    "organize organized organizer organization"
]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

features = [
    stemmer.stem(word)
    for word in vectorizer.get_feature_names_out()
]
```

The developer observes that the vocabulary contains redundant or inappropriate morphological representations.

Tasks

1. Identify the preprocessing error in the program.
2. Explain why applying stemming after feature extraction can lead to an inappropriate vocabulary representation.
3. Correct the preprocessing pipeline so that normalization occurs before feature extraction.
4. Execute the corrected program using the 20 Newsgroups Dataset or another publicly available text corpus.

5. **Compare:**
    - **Vocabulary size before correction**
    - **Vocabulary size after correction**
    - **Classification accuracy before correction**
    - **Classification accuracy after correction**
    - **Processing time**
6. **Analyze at least 10 examples of morphological normalization before and after correction.**
7. **Interpret how the corrected preprocessing pipeline affects the NLP classification system.**

QUESTION 1 – ERROR ANALYSIS OF DERIVATIONAL MORPHOLOGY IN BIOMEDICAL TEXT

1. Identify words that produce incorrect morphological analyses

The selected words are treatment, treatable, retreatment, treated, and untreated. The system produces errors when it removes prefixes or suffixes without considering their morphological meaning.

2. Decompose the words into prefixes, roots, and suffixes

Word	Prefix	Root	Suffix
Treatment	—	treat	-ment
Treatable	—	treat	-able
Retreatment	re-	treat	-ment
Treated	—	treat	-ed
Untreated	un-	treat	-ed

3. Identify whether each affix is inflectional or derivational

The prefixes re- and un- and the suffixes -ment and -able are derivational. The suffix -ed is inflectional when it represents past tense.

4. Explain the root cause of the incorrect morphological analysis

The main cause is that the system removes affixes without distinguishing between derivational and inflectional morphology. As a result, important semantic information may be lost.

5. Propose a corrected morphological-analysis strategy

The system should identify the prefix, root, and suffix, classify each affix, and then decide whether it should be removed or retained. A biomedical vocabulary can also be used to validate the analysis.

6. Compare the original and corrected analyses

- Treatment → treat + ment
- Treatable → treat + able
- Retreatment → re + treat + ment
- Treated → treat + ed
- Untreated → un + treat + ed

7. Explain how the correction improves biomedical information retrieval

Correct morphological analysis connects related terms while preserving important meaning. This improves indexing, search accuracy, and retrieval of relevant biomedical articles.

QUESTION 2 – ERROR ANALYSIS OF FINITE-STATE MORPHOLOGICAL PARSING

1. Identify words that produce incorrect morphological analyses

The selected words are replayed, unhappier, disconnected, players, restarting, and unreadable. The parser mainly fails when multiple affixes occur in the same word.

2. Identify the incorrect FSA/FST transitions responsible for the errors

The parser may recognize only one affix and stop before processing the remaining affixes.

For example:

replayed → re + play + ed

Similarly:

- unhappier → un + happy + er
- disconnected → dis + connect + ed
- players → play + er + s

The error occurs because the finite-state transitions do not allow multiple affixes.

3. Modify the finite-state transitions

The improved structure should allow multiple prefixes and suffixes:

START → Prefix → Root → Suffix → END**

Correct analyses include:

- replayed → re + play + ed
- unhappier → un + happy + er
- disconnected → dis + connect + ed
- players → play + er + s
- restarting → re + start + ing
- unreadable → un + read + able

4. Execute the corrected parser

Word	Correct Analysis
Replayed	re + play + ed
Unhappier	un + happy + er
Disconnected	dis + connect + ed
Players	play + er + s
Restarting	re + start + ing
Unreadable	un + read + able

5. Compare the parser output before and after correction

Before correction, the parser recognizes only one affix or stops early. After correction, it recognizes multiple prefixes and suffixes correctly.

6. Calculate parsing accuracy

The formula is:

Accuracy = (Correct Analyses / Total Analyses) × 100

For example, if 3 out of 6 analyses are correct before correction:

Accuracy = (3/6) × 100 = 50%

If all 6 are correct after correction:

Accuracy = (6/6) × 100 = 100%

7. Limitations and computational complexity

Finite-state parsing is efficient and generally works in O(n) time for a word of length *n*. However, it requires manually defined rules and may have difficulty handling irregular words, spelling variations, and new social-media words.

QUESTION 3 – ERROR ANALYSIS OF A STEMMING PROGRAM

1. Identify the programming and preprocessing errors

The main error is:

data["Processed"] = data["Text"].apply(ps.stem)

PorterStemmer expects individual words, not complete sentences. The program also does not perform tokenization before stemming.

2. Explain why the output is incorrect

The program directly passes an entire sentence to the stemmer. A sentence must first be divided into individual words, and then each word should be stemmed separately.

3. Correct the program

```
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize
import pandas as pd

ps = PorterStemmer()

data = pd.read_csv("news.csv")

def preprocess(text):
    tokens = word_tokenize(str(text).lower())
    return [ps.stem(word) for word in tokens]

data["Tokens"] = data["Text"].apply(
    lambda x: word_tokenize(str(x).lower())
)

data["Stemmed"] = data["Text"].apply(preprocess)
print(data[["Text", "Tokens", "Stemmed"]].head())
```

4. Execute the corrected program using a news corpus

Example:

Original text:

The researchers studied new treatments.

Tokens:

the, researchers, studied, new, treatments

Stemmed tokens:

the, research, studi, new, treatment

5. Compare original text, tokens, and stemmed tokens

Original Text	Tokens	Stemmed Tokens
Researchers studied treatments	researchers, studied, treatments	research, studi, treatment
Players played well	players, played, well	player, play, well

6. Identify 20 problematic stemming cases

Word	Stem	Type
Studies	studi	Inflectional
Studied	studi	Inflectional
Studying	studi	Inflectional
Cats	cat	Inflectional
Dogs	dog	Inflectional
Readable	readabl	Derivational
Happiness	happi	Derivational
Kindness	kind	Derivational
Quickly	quickli	Derivational
National	nation	Derivational
Development	develop	Derivational

7. Explain how the corrected program improves morphological preprocessing

The corrected program tokenizes sentences before stemming. This provides better word-level processing, morphological normalization, feature extraction, search, and classification.

QUESTION 4 – ERROR ANALYSIS OF A FINITE-STATE MORPHOLOGICAL PARSER

1. Identify all logical errors in the program

The main error is the use of:

word[:-2]

This removes two characters even though a regular plural normally requires removing only the final s.

The program also does not handle:

- -es plurals
- -ies plurals
- Irregular plurals

2. Explain why the parser fails for different plural formation rules

English uses different plural formation rules:

- car → cars
- box → boxes
- city → cities
- child → children

The original parser handles only a simple plural ending in -s.

3. Modify the program

```
def parser(word):

    irregular = {
        "children": "child",
        "men": "man",
        "women": "woman",
        "mice": "mouse"
    }
    if word in irregular:
        return irregular[word], "Plural Noun"
    elif word.endswith("ies"):
        return word[:-3] + "y", "Plural Noun"
    elif word.endswith("es"):
        return word[:-2], "Plural Noun"
    elif word.endswith("s"):
        return word[:-1], "Plural Noun"
    else:
        return word, "Singular"

words = ["cars", "boxes", "cities", "children", "books"]

for w in words:
    print(w, "->", parser(w))
```

4. Execute the corrected parser

Word	Original Output	Corrected Output
Cars	car, Plural	car, Plural
Boxes	boxe, Plural	box, Plural
Cities	citie, Plural	city, Plural
Children	children, Singular	child, Plural
Books	book, Plural	book, Plural

5. Compare the original and corrected outputs

The original parser incorrectly removes two characters from words ending in s. The corrected parser checks irregular, -ies, -es, and regular -s forms.

6. Identify the limitations of the rule-based approach

The parser cannot handle every irregular word, may incorrectly process some words ending in s, does not use sentence context, and requires manually defined rules.

7. Explain how the corrected parser improves morphological analysis

The corrected parser recognizes more English plural patterns and produces better root forms such as:

cities → city

boxes → box

children → child

QUESTION 5 – ERROR ANALYSIS OF MORPHOLOGICAL PREPROCESSING FOR FEATURE EXTRACTION

1. Identify the preprocessing error in the program

The main error is that stemming is performed after CountVectorizer creates the vocabulary.

Original pipeline:

Documents → CountVectorizer → Vocabulary → Stemming

2. Explain why this creates an inappropriate vocabulary

CountVectorizer creates separate features for different word forms before stemming. Therefore, words such as connected, connection, and connecting may remain as separate features.

3. Correct the preprocessing pipeline

The correct pipeline is:

Documents → Tokenization → Stemming → CountVectorizer → Features

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
```

```
    "connected connection connecting connectivity",
    "studies studied studying study",
    "organize organized organizer organization"
]

def stem_text(text):
    return " ".join(
        stemmer.stem(word)
        for word in text.lower().split()
    )

processed = [stem_text(doc) for doc in documents]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(processed)

print(vectorizer.get_feature_names_out())
```

4. Execute the corrected program using a text corpus

The same pipeline can be applied to the 20 Newsgroups Dataset:

Raw Text → Stemming → Vectorization → Classification

The exact vocabulary size, accuracy, and processing time depend on the dataset, classifier, and experimental setup.

5. Compare the results

Measure	Before Correction	After Correction
Vocabulary size	Usually larger	Usually smaller
Redundancy	High	Reduced
Features	Separate forms	Normalized forms
Processing	Stemming after extraction	Stemming before extraction

6. Analyze morphological normalization examples

Original	Normalized	Type
Connected	connect	Inflectional
Connecting	connect	Inflectional
Connection	connect	Derivational
Studies	studi	Inflectional
Studied	studi	Inflectional
Studying	studi	Inflectional
Organized	organ	Inflectional
Organizer	organ	Derivational

Organization	organ	Derivational
Developed	develop	Inflectional

7. Interpret the effect on the classification system

The corrected pipeline reduces redundant features and creates a more compact vocabulary. This can reduce memory usage and improve training efficiency. However, excessive stemming may remove useful meaning, so classification accuracy should be measured experimentally.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1	4	4	4	4	3	19	94
2	3	4	4	4	4	19	
3	4	4	4	4	3	19	
4	3	4	4	4	4	19	
5	4	4	4	3	4		

Faculty In-charge	Course Coordinator	Program Director
Dr.T.Kumaragurubaran		
Signature: ________________	Signature: ________________	Signature: ________________
Date: ____________________	Date: ____________________	Date: ____________________